

CS 3502

Operating Systems

Project 1

Kun Suo

Computer Science, Kennesaw State University

<https://kevinsuo.github.io/>

Command: fish

- <https://fishshell.com/>

Autosuggestions

fish suggests commands as you type based on history and completions, just like a web browser. Watch out, Netscape Navigator 4.0!

```
tion to do it  
nber it and s  
it.  
> sh -l den
```

Sane Scripting

fish is fully scriptable, and its syntax is simple, clean, and consistent. You'll never write esac again.

```
function de  
switch  
cas  
cas
```

Man Page Completions

Other shells support programmable completions, but only fish generates them automatically by parsing your installed man pages.

```
demo > fish_  
Parsing man  
demo > rwsno  
-P (print p  
-Z  
-j (
```

Glorious VGA Color

fish supports 24 bit true color, the state of the art in terminal technology. Behold the monospaced rainbow.

```
mo > echo "Lc  
mo > █
```

Web Based configuration

For those lucky few with a graphical computer, you can set your colors and view functions, variables, and history all from a web page.

```
Current  
/bright/vi  
echo 'Erro  
# This is  
Th is an
```

Works Out Of The Box

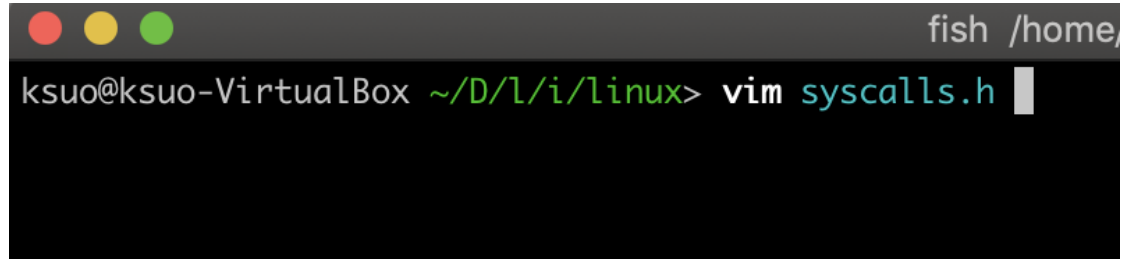
fish will delight you with features like tab completions and syntax highlighting that just work, with nothing new to learn or configure.

```
git checkout  
(Tag) 1.21  
(Tag) 1.21  
(Tag) 1.21  
(Tag) 1.21  
(Tag) 1.21
```

Project 1: No IDE, please use vim

- Open a file

- `$ vim test.c`



```
fish /home/ksuo@ksuo-VirtualBox ~/D/l/i/linux> vim syscalls.h
```

- Close a file

- Inside vim (after opening), press

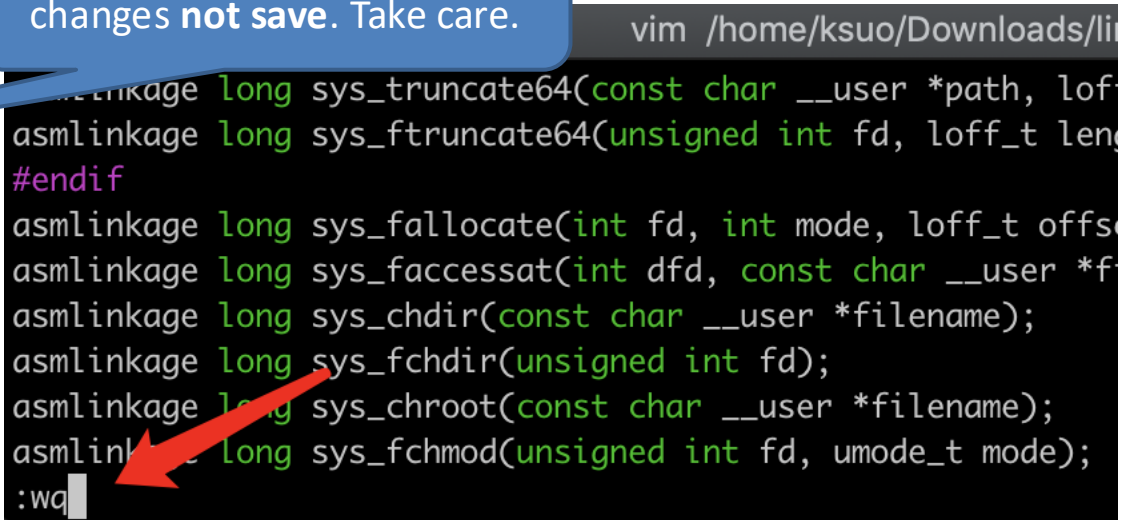
`:q!`

- Inside vim (after opening), press

`:wq`, changes saved

colon mark means menu.

Exclamation mark means changes **not** save. Take care.

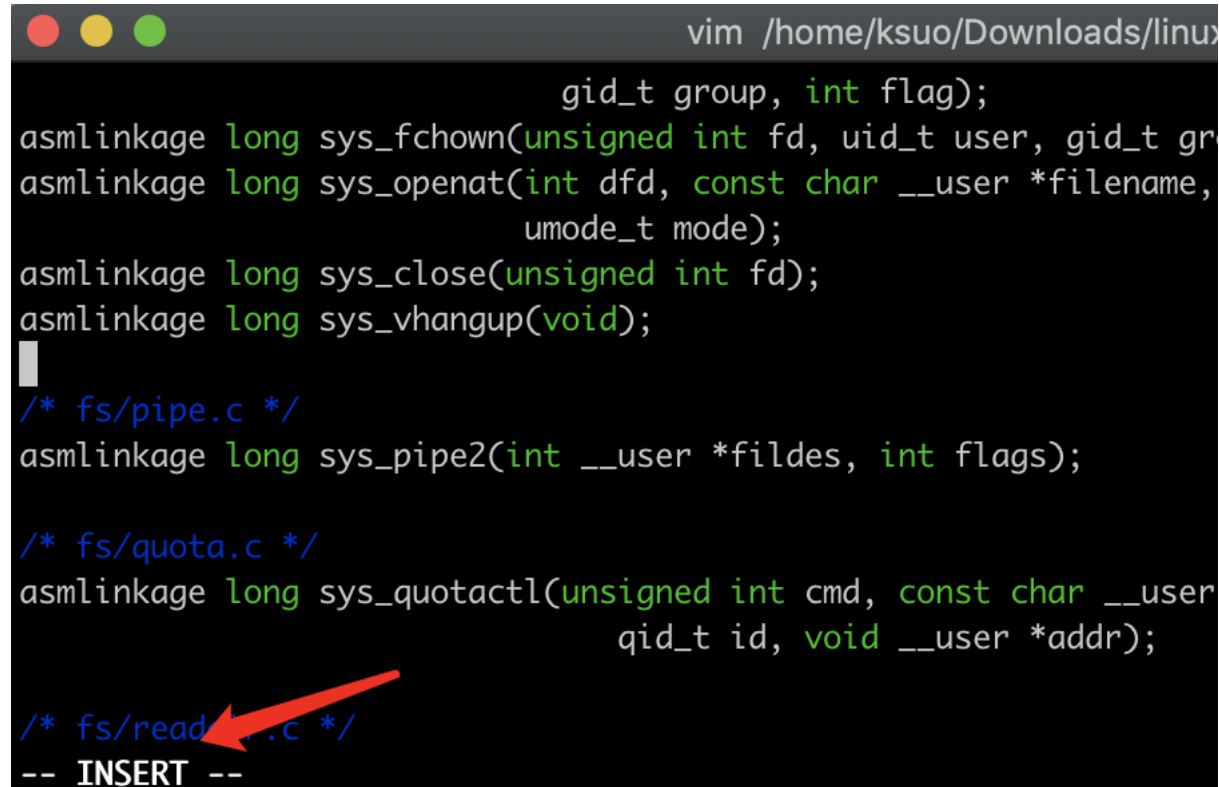


```
vim /home/ksuo/Downloads/li
asmlinkage long sys_truncate64(const char __user *path, loff_t len, mode_t mode);
asmlinkage long sys_ftruncate64(unsigned int fd, loff_t length, mode_t mode);
#endif
asmlinkage long sys_fallocate(int fd, int mode, loff_t offset, loff_t length);
asmlinkage long sys_faccessat(int dfd, const char __user *filename, int mode, int flags);
asmlinkage long sys_chdir(const char __user *filename);
asmlinkage long sys_fchdir(unsigned int fd);
asmlinkage long sys_chroot(const char __user *filename);
asmlinkage long sys_fchmod(unsigned int fd, umode_t mode);
:wq
```



Project 1: No IDE, please use vim

- Edit a file
 - Open a file
 - Press i (means enter the insert mode). then input your code
 - Press ESC to exit the insert mode
 - Close a file (:wq)



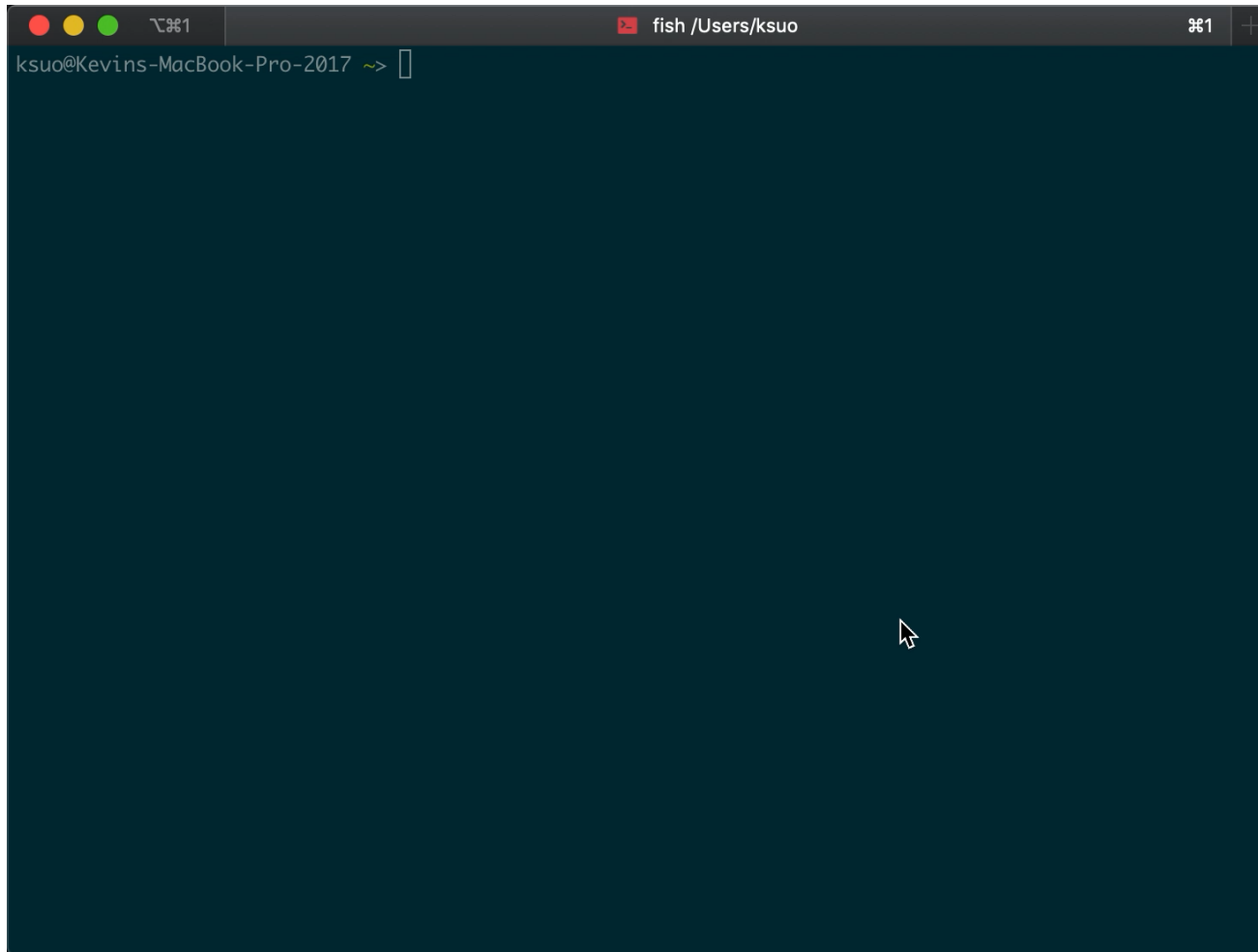
```
vim /home/ksuo/Downloads/linux
                                gid_t group, int flag);
asmlinkage long sys_fchown(unsigned int fd, uid_t user, gid_t gr
asmlinkage long sys_openat(int dfd, const char __user *filename,
                                umode_t mode);
asmlinkage long sys_close(unsigned int fd);
asmlinkage long sys_vhangup(void);
█
/* fs/pipe.c */
asmlinkage long sys_pipe2(int __user *fildes, int flags);

/* fs/quota.c */
asmlinkage long sys_quotactl(unsigned int cmd, const char __user
                                gid_t id, void __user *addr);

/* fs/read.c */
-- INSERT --
```

Project 1: No IDE, please use vim

<https://youtu.be/rS0XZpLtVlo>



Edit a file with vim

- step 1: \$ **vim** file
- step 2: press **i**, enter insert mode; move the cursor to position and edit the context
- step 3: after editing, press **ESC** to exit the insert mode to normal mode
- step 4: press **:wq** to save what you edit and quit. If you do not want to save, press **:q!**



More about vim

- A quick start guide for beginners to the Vim text editor
 - <https://eastmanreference.com/a-quick-start-guide-for-beginners-to-the-vim-text-editor>
- Vim basics:
 - <https://www.howtoforge.com/vim-basics>
- Learn the Basic Vim Commands [Beginners Guide]
 - https://www.youtube.com/watch?time_continue=265&v=ZEGqkam-3lc



Outline

- **Part A: create a VM and compile your kernel**
- Part B/C: add a new system call into the Linux kernel



How to build one ubuntu VM?

- HostOS

Windows 10 (x86):

<https://www.youtube.com/watch?v=QbmRXJJKsvs>

MacOS (x86):

<https://www.youtube.com/watch?v=GDoCrfPma2k&t=321s>

MacOS (arm):

<https://youtu.be/O19mv1pe76M?si=4cYayFiqPNoHoY1w>



Tips

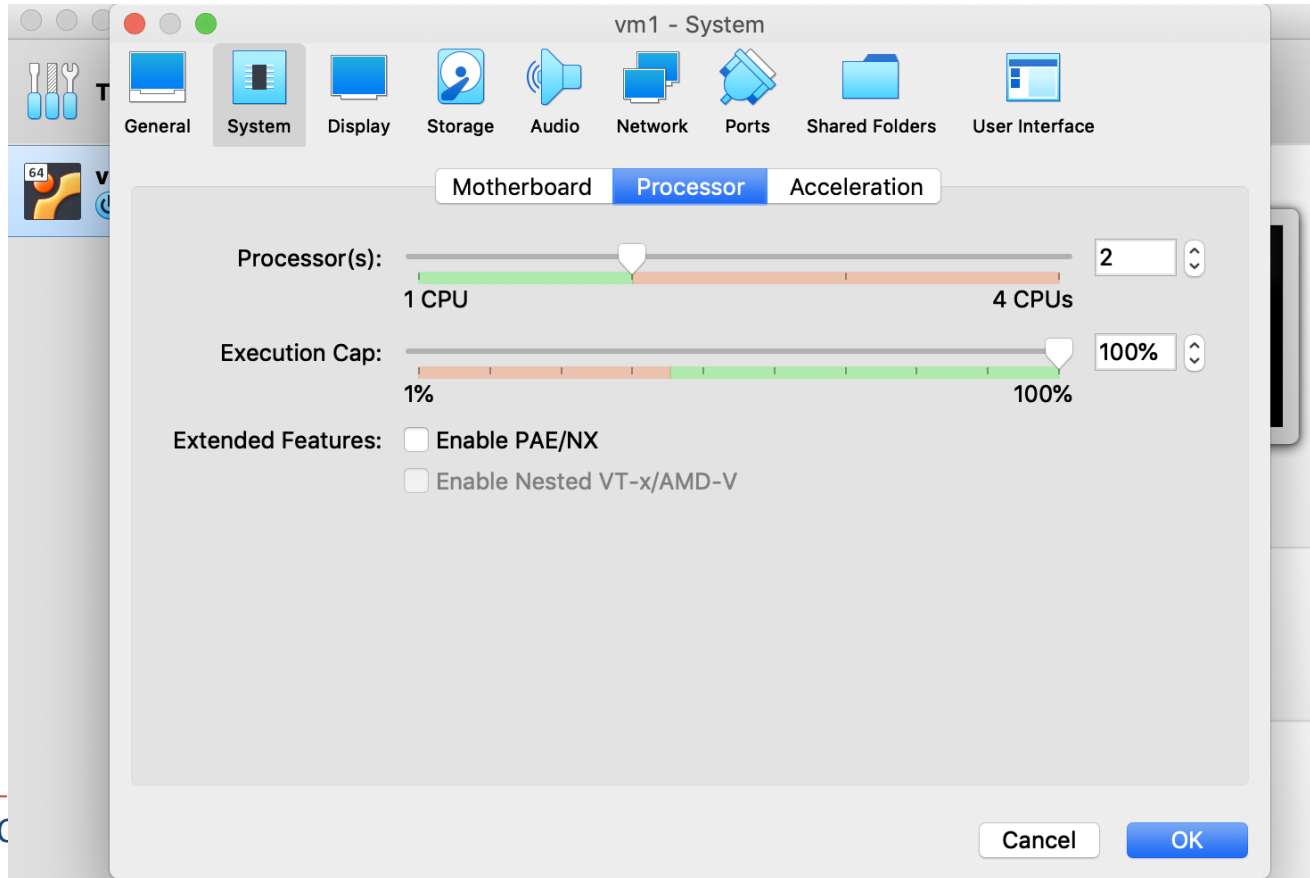
- Compile Linux kernel takes half to two hours, depending on machine speed
- To save the time, use `$ make localmodconfig`
- Run many commands in one:

`$ sudo make; sudo make modules; sudo make modules_install; sudo make install`

```
ksuo@ksuo-VirtualBox ~/D/linux-5.1>  
sudo make; sudo make modules; sudo make modules_install; sudo make install
```

Tips

- Set up more CPUs for VM
- Make `-j N`, to accelerate compiling (N is your CPU number)



Where is my kernel?

- `$ ls /boot/`

Initial ramdisk: loading a temporary root file system into memory. Used for startup.

Linux executable kernel image

initrd.img-5.0.0-23-generic
initrd.img-5.0.0-25-generic
initrd.img-5.1.0

vmlinuz-5.0.0-23-generic
vmlinuz-5.0.0-25-generic
vmlinuz-5.1.0

Which kernel to boot if there are many?

If you are using Ubuntu: change the grub configuration file:

```
$ sudo vim /etc/default/grub
```

The OS boots by using the first kernel by default. You have 10 seconds to choose.

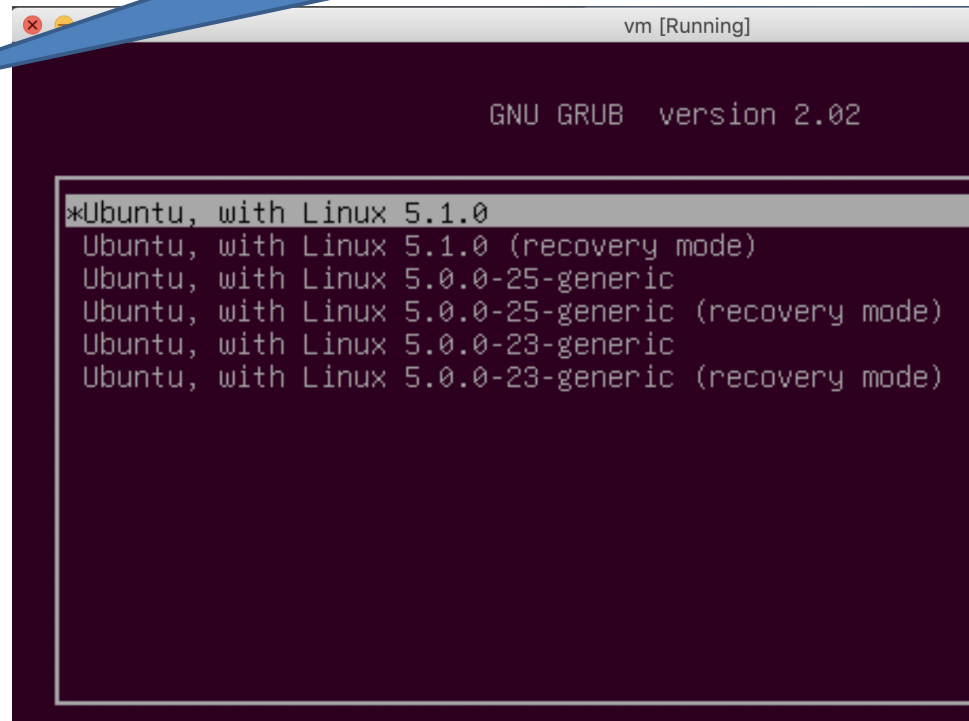
Make the following changes:

```
GRUB_DEFAULT=0
```

```
GRUB_TIMEOUT=10
```

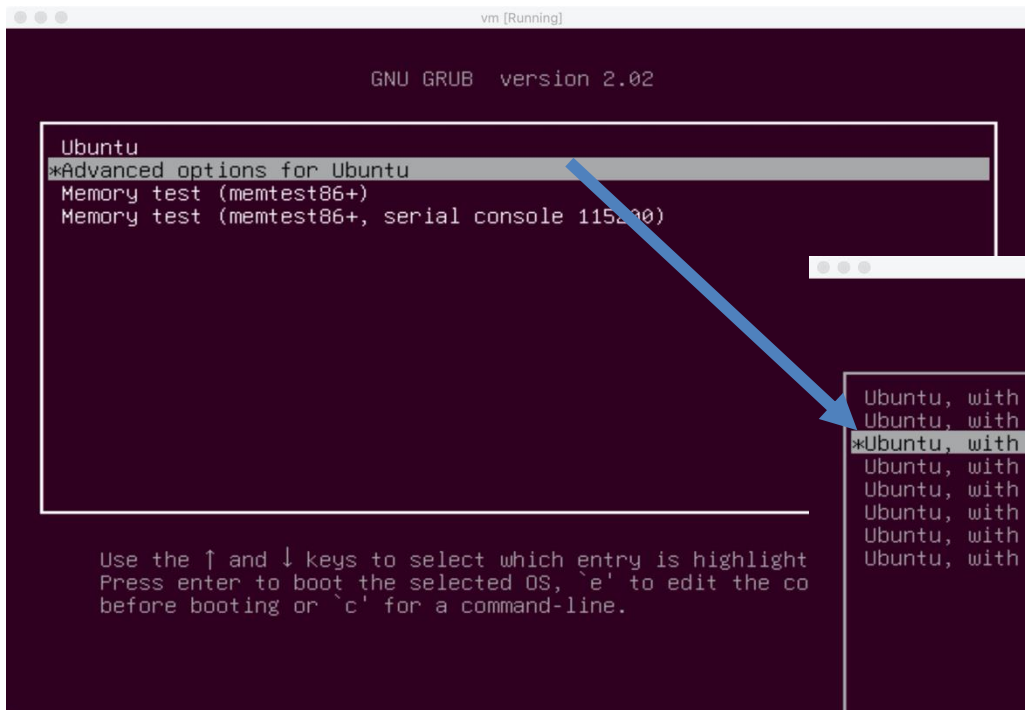
Then, update the grub entry:

```
$ sudo update-grub2
```



Which kernel to boot if there are many?

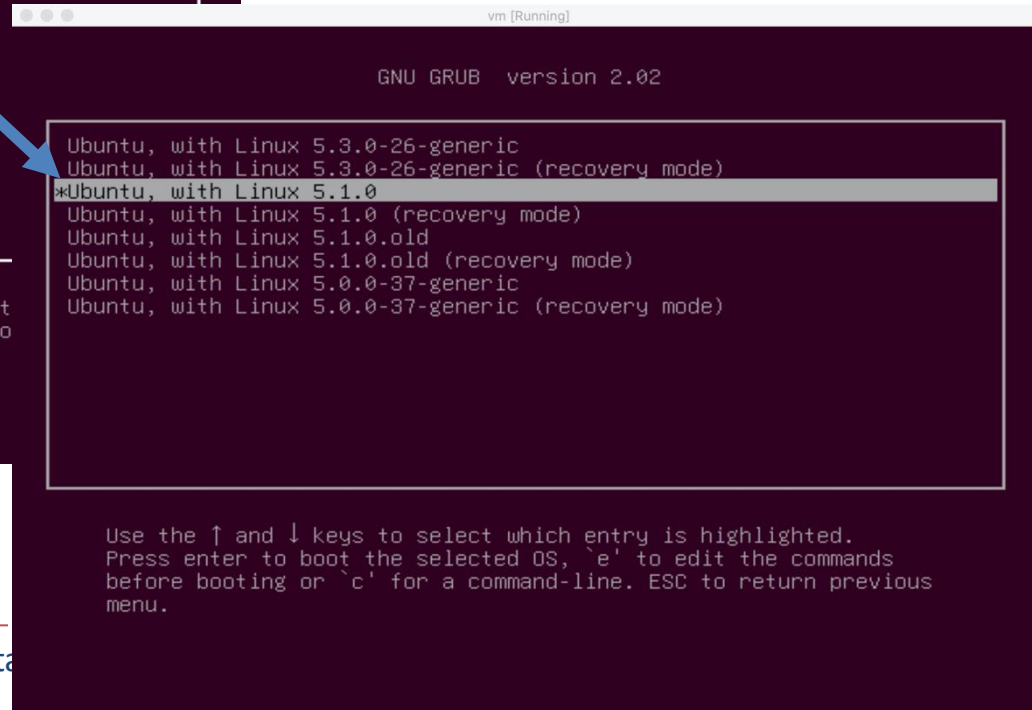
Immediately after the BIOS/UEFI splash screen during boot, with BIOS, quickly press and hold the **Shift** key, which will bring up the GNU GRUB menu. (If you see the Ubuntu logo, you've missed the point where you can enter the GRUB menu.)



```
vm [Running]
GNU GRUB version 2.02

Ubuntu
*Advanced options for Ubuntu
Memory test (memtest86+)
Memory test (memtest86+, serial console 115200)
```

Use the ↑ and ↓ keys to select which entry is highlighted.
Press enter to boot the selected OS, 'e' to edit the command before booting or 'c' for a command-line.



```
vm [Running]
GNU GRUB version 2.02

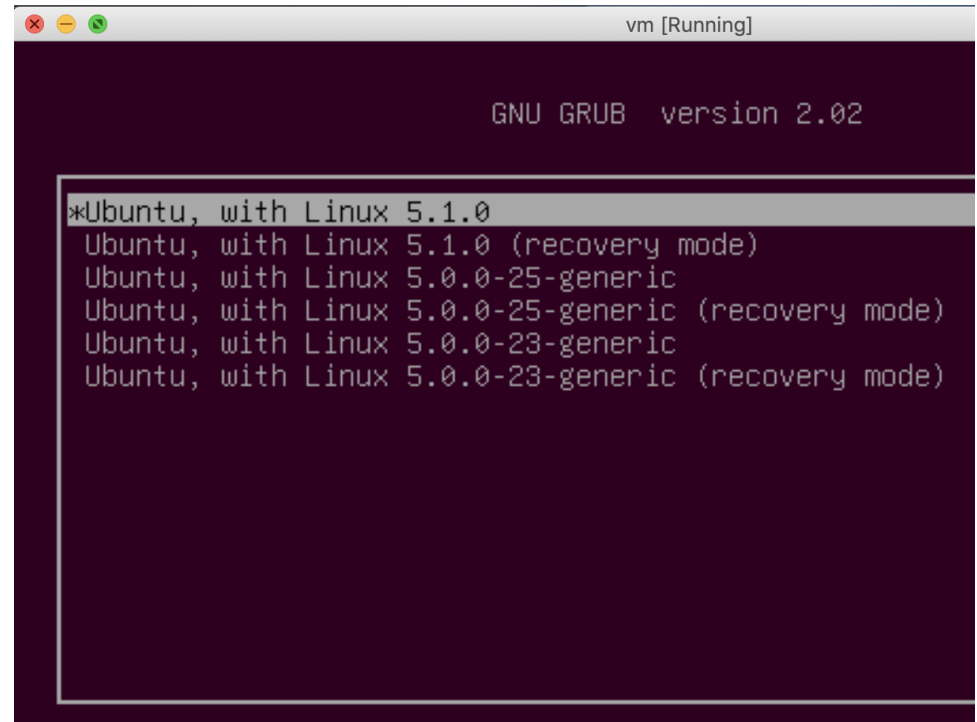
Ubuntu, with Linux 5.3.0-26-generic
Ubuntu, with Linux 5.3.0-26-generic (recovery mode)
*Ubuntu, with Linux 5.1.0
Ubuntu, with Linux 5.1.0 (recovery mode)
Ubuntu, with Linux 5.1.0.old
Ubuntu, with Linux 5.1.0.old (recovery mode)
Ubuntu, with Linux 5.0.0-37-generic
Ubuntu, with Linux 5.0.0-37-generic (recovery mode)
```

Use the ↑ and ↓ keys to select which entry is highlighted.
Press enter to boot the selected OS, 'e' to edit the commands before booting or 'c' for a command-line. ESC to return previous menu.



What if my kernel crashed?

- Your kernel could crash because you might bring in some kernel bugs
- In the menu, choose the old kernel to boot the system
- Fix your bug in the source code
- Compile and reboot

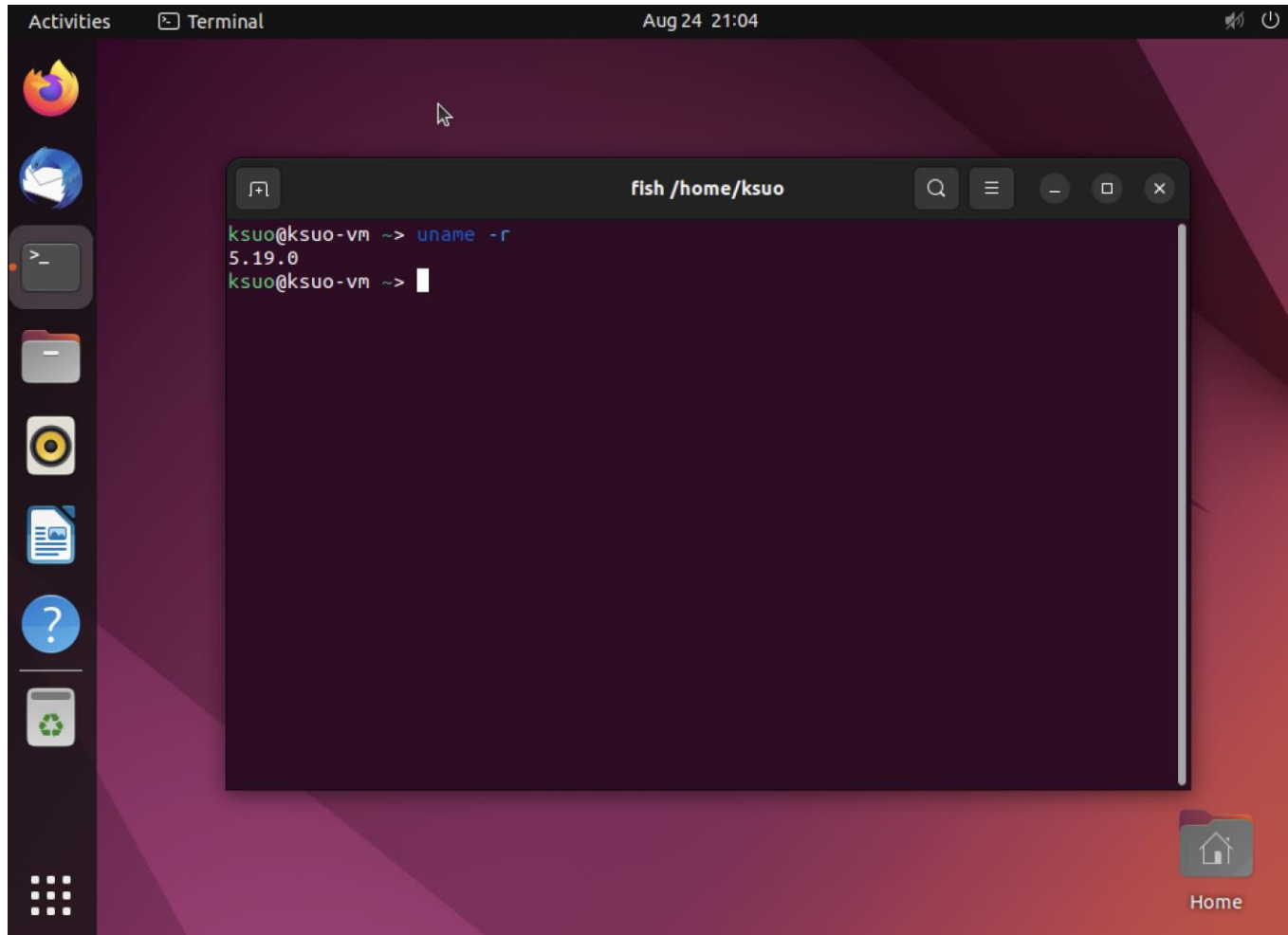


Compile the kernel

1. download the kernel source code
2. unzip the file
3. use *\$ make localmodconfig* to create a config file
4. use *\$ make/make modules* to compile the code
5. use *\$ make modules_install/make install* to install the kernel
6. reboot the VM



Screenshot of my new kernel



Outline

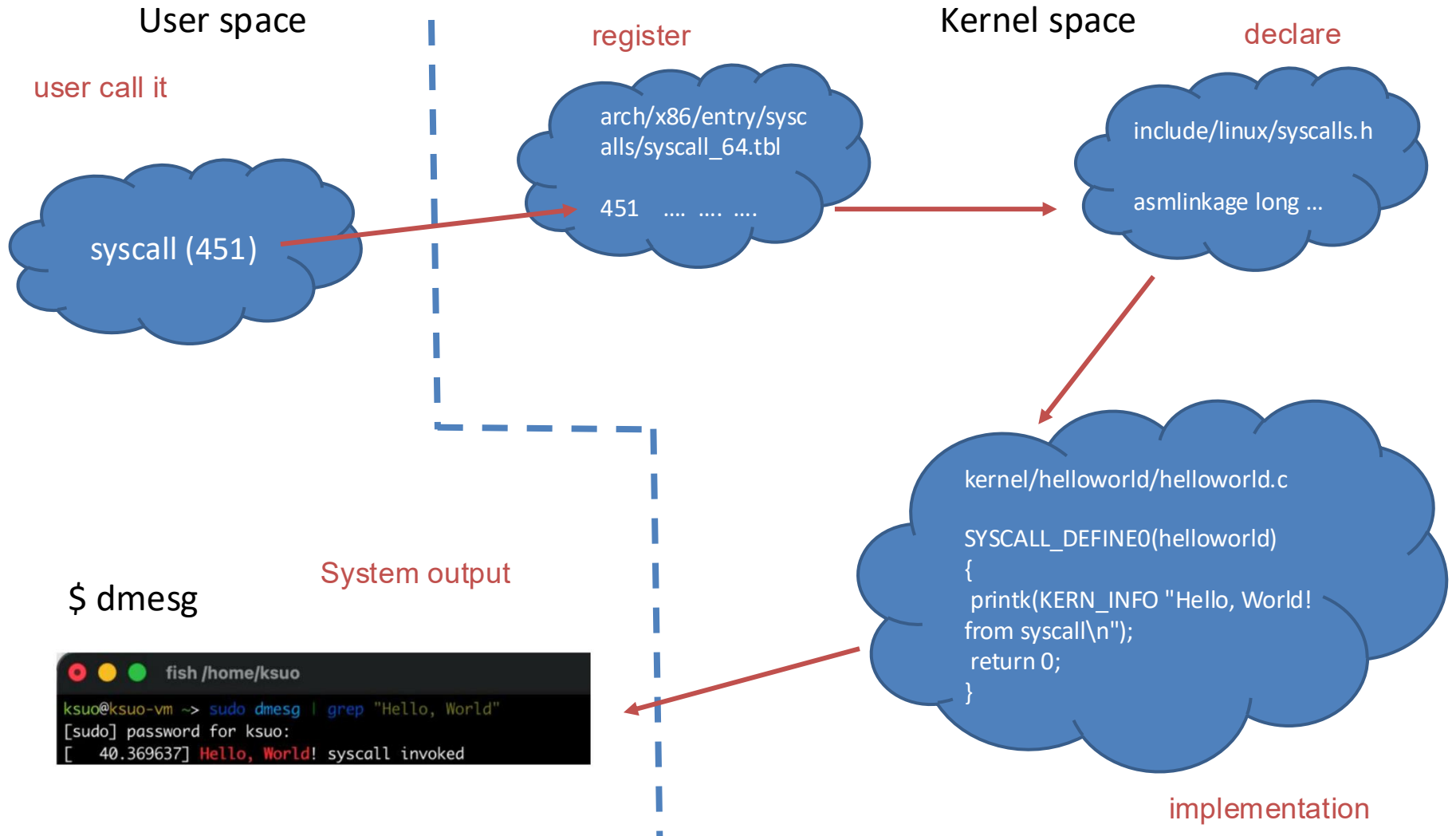
- Part A: create a VM and compile your kernel
- **Part B/C: add a new system call into the Linux kernel**



Part B: Define your system call (e.g., x86)

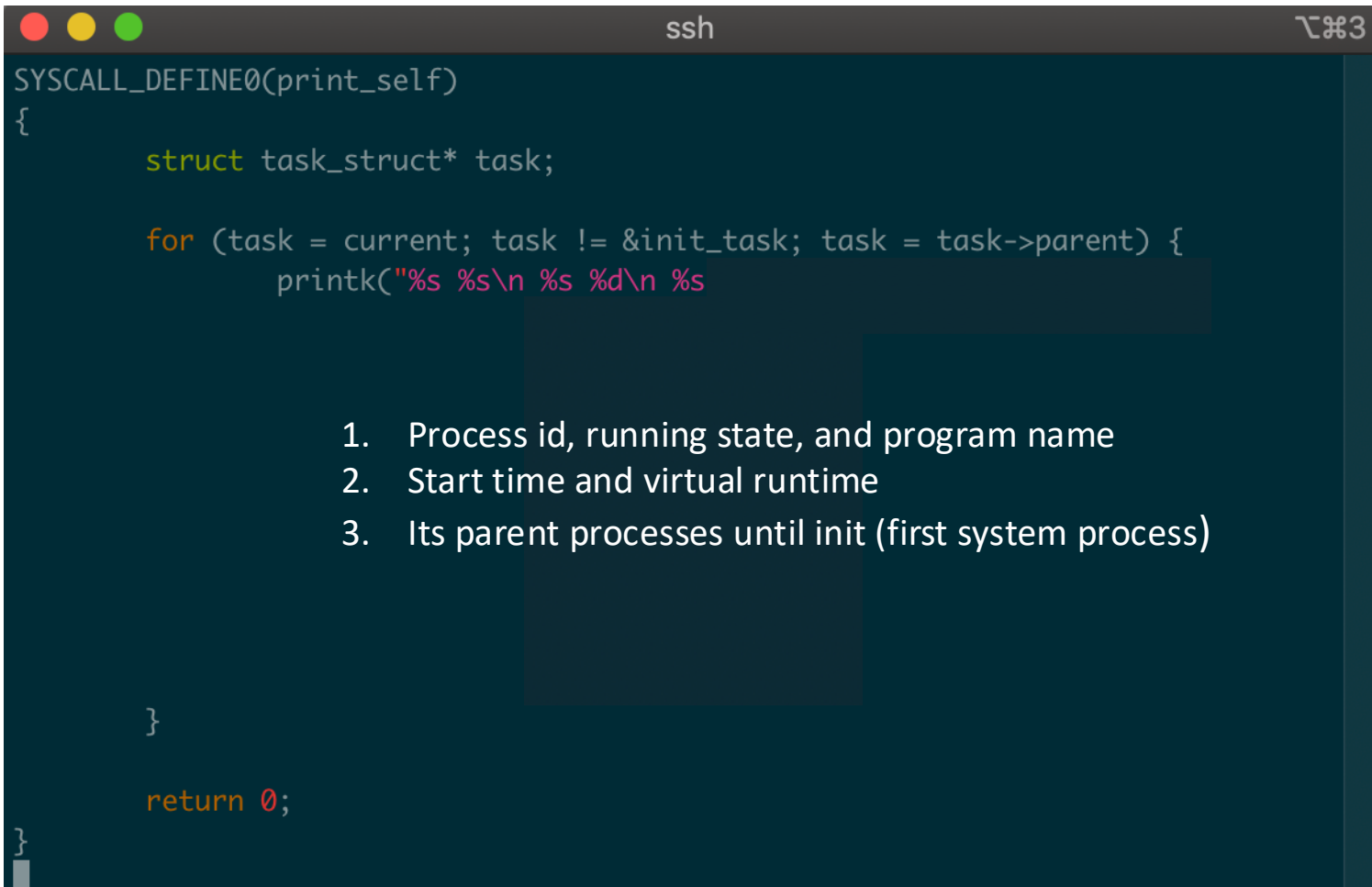
- Step 1: Add the System Call Implementation
- Step 2: Register the Syscall in the Syscall Table
- Step 3: Declare the Syscall in the Header
- Step 4: Hook Into the Build System
- Step 5: write a user-level program to test your system call

Part B: Put it all together



Part C: implementation

Kernel function
structure



```
ssh \#3
SYSCALL_DEFINE0(print_self)
{
    struct task_struct* task;

    for (task = current; task != &init_task; task = task->parent) {
        printk("%s %s\n %s %d\n %s

1. Process id, running state, and program name
2. Start time and virtual runtime
3. Its parent processes until init (first system process)

    }

    return 0;
}
```



Part C: implementation

- print out the following information of the calling process:
 - Process id, running state, and program name
 - Start time and virtual runtime
 - Its parent processes until init (first system process)



Part C: implementation

- print out the following information of the calling process:
 - **Process id**, running state, and program name
 - Start time and virtual runtime
 - Its parent processes until init (first system process)

<https://elixir.bootlin.com/linux/v5.1/source/include/linux/sched.h#L739>



Part C: implementation

- print out the following information of the calling process:
 - Process id, running state, and program name
 - Start time and virtual runtime
 - Its parent processes until init (first system process)

<https://elixir.bootlin.com/linux/v5.1/source/include/linux/sched.h#L594>



Part C: implementation

- print out the following information of the calling process:
 - Process id, running state, and program name
 - Start time and virtual runtime
 - Its parent processes until init (first system process)

<https://elixir.bootlin.com/linux/v5.1/source/include/linux/sched.h#L840>



Part C: implementation

- print out the following information of the calling process:
 - Process id, running state, and program name
 - Start time and virtual runtime
 - Its parent processes until init (first system process)

<https://elixir.bootlin.com/linux/v5.1/source/include/linux/sched.h#L808>



Part C: implementation

- print out the following information of the calling process:
 - Process id, running state, and program name
 - Start time and virtual runtime
 - Its parent processes until init (first system process)

<https://elixir.bootlin.com/linux/v5.1/source/include/linux/sched.h#L637>

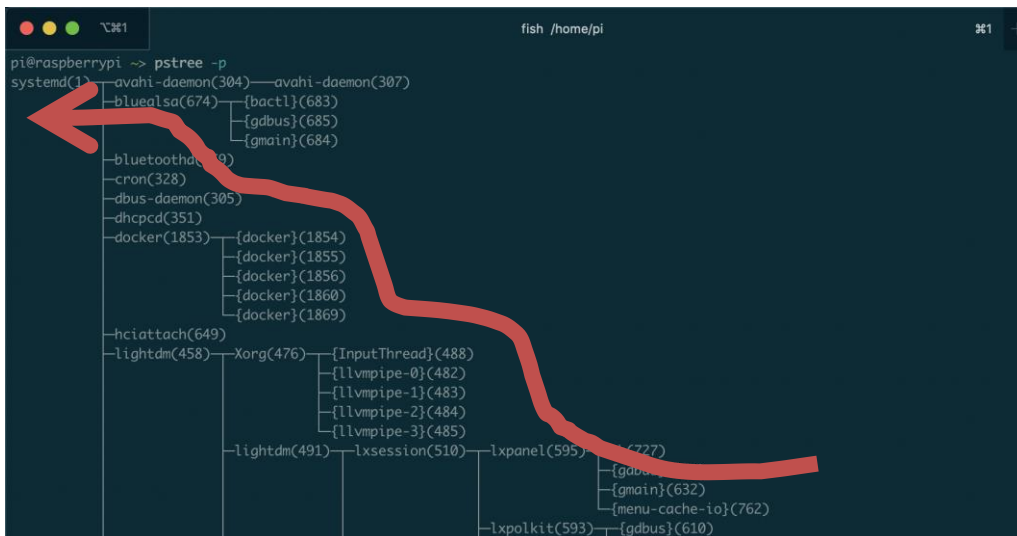
<https://elixir.bootlin.com/linux/v5.1/source/include/linux/sched.h#L450>

Task_struct
→ sched_entity
→ vruntime



Part C: implementation

- print out the following information of the calling process:
 - Process id, running state, and program name
 - Start time and virtual runtime
 - Its **parent processes** until init (first system process)

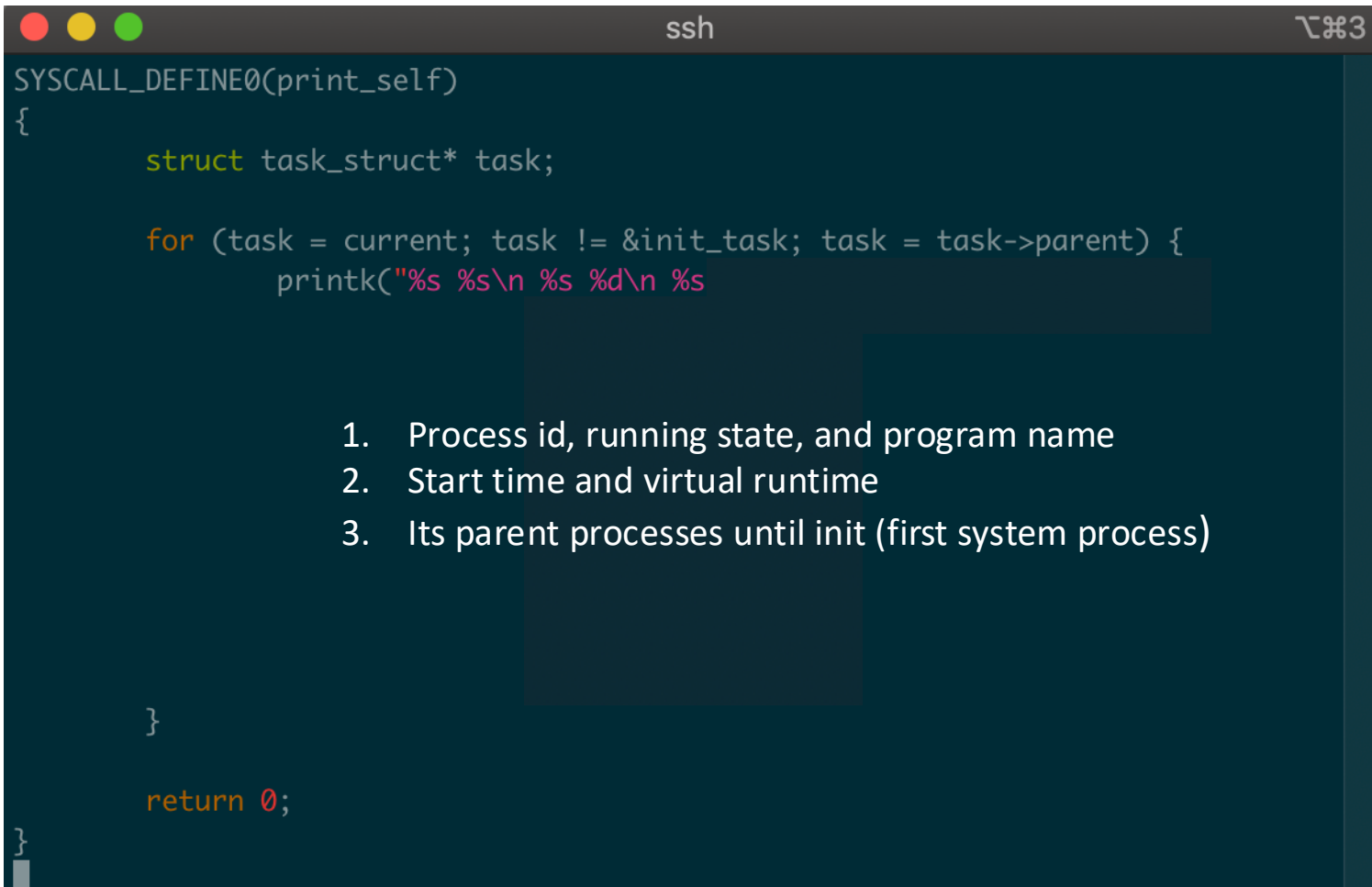


For loop until it reaches the init

```
{
    printk(...)
}
```

Part C: implementation

Kernel function
structure



```
ssh \#3
SYS_CALL_DEFINE0(print_self)
{
    struct task_struct* task;

    for (task = current; task != &init_task; task = task->parent) {
        printk("%s %s\n %s %d\n %s

1. Process id, running state, and program name
2. Start time and virtual runtime
3. Its parent processes until init (first system process)

    }

    return 0;
}
```



At the end of output of \$dmesg

```
[ 66.439077] Take name: test2
Task PID: 1714
Task State: 0
Virtual runtime: 95781496
Start time: 66428984292

[ 66.439079] Take name: sudo
Task PID: 1713
Task State: 1
Virtual runtime: 49524624
Start time: 65811247276

[ 66.439081] Take name: fish
Task PID: 1553
Task State: 1
Virtual runtime: 86586173
Start time: 15521078146

[ 66.439082] Take name: bash
Task PID: 1542
Task State: 1
Virtual runtime: 29803021
Start time: 14425010066

[ 66.439083] Take name: sshd
Task PID: 1541
Task State: 1
Virtual runtime: 235926572
Start time: 14421565758

[ 66.439084] Take name: sshd
Task PID: 1454
Task State: 1
Virtual runtime: 280309726
Start time: 13210824978

[ 66.439086] Take name: sshd
Task PID: 693
Task State: 1
Virtual runtime: 6283579
Start time: 6391273714

[ 66.439087] Take name: systemd
Task PID: 1
Task State: 1
Virtual runtime: 252410194
Start time: 211561921

[ 66.439082] Take name: bash
Task PID: 1542
Task State: 1
Virtual runtime: 29803021
Start time: 14425010066

[ 66.439083] Take name: sshd
Task PID: 1541
Task State: 1
Virtual runtime: 235926572
Start time: 14421565758

[ 66.439084] Take name: sshd
Task PID: 1454
Task State: 1
Virtual runtime: 280309726
Start time: 13210824978

[ 66.439086] Take name: sshd
Task PID: 693
Task State: 1
Virtual runtime: 6283579
Start time: 6391273714

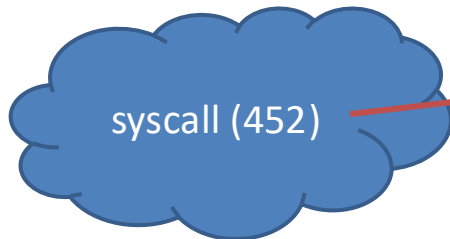
[ 66.439087] Take name: systemd
Task PID: 1
Task State: 1
Virtual runtime: 252410194
Start time: 211561921
```

ksuo@ksuo-VirtualBox ~>

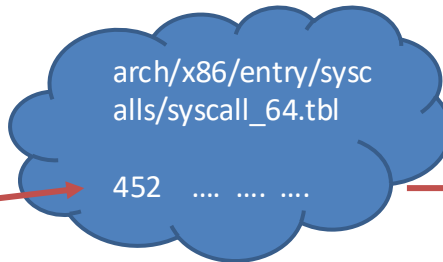
Part C: Put it all together

User space

user call it

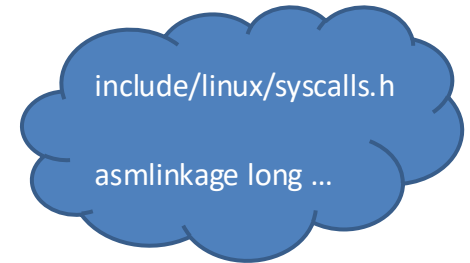


register



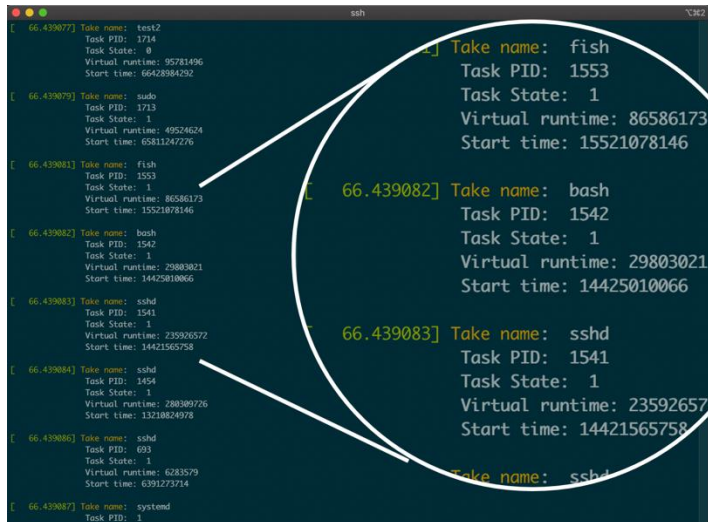
Kernel space

declare

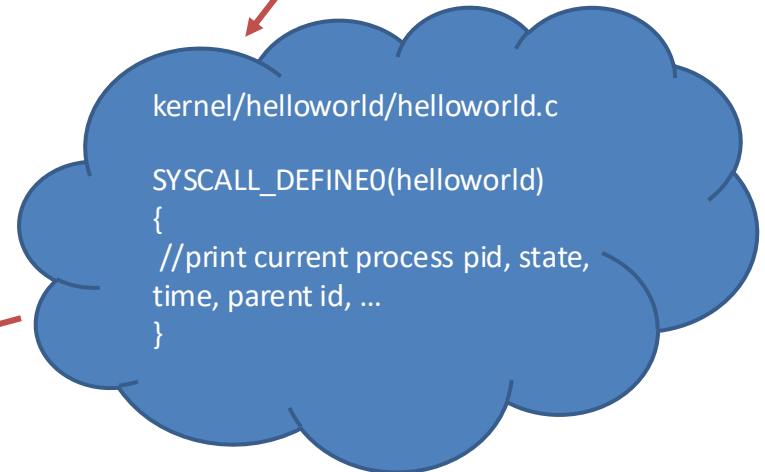


\$ dmesg

System output



A terminal window showing the output of the "dmesg" command. The output lists system events with details like "Take name", "Task PID", "Task State", "Virtual runtime", and "Start time". A white circle highlights a specific event: "66.439082] Take name: bash Task PID: 1542 Task State: 1 Virtual runtime: 29803021 Start time: 1442501006".



implementation